

MediBridge — Full Architecture & Codebase Documentation

A Comprehensive Developer Guide, File-by-File Reference, API Map & Modification Manual

1. Project Overview

MediBridge is an end-to-end intelligent telemedicine and clinical records management platform designed to connect patients and specialized healthcare providers seamlessly. Built as a decoupled web application with a **React (Vite)** frontend and a **Python (Flask)** backend, MediBridge incorporates advanced artificial intelligence for patient triage, doctor matching, clinical report synthesis, optical character recognition (OCR), and assistive voice capabilities.

Core Pillars:

- **Patient Portal:** Allows patients to browse specialist directories manually or describe health symptoms in plain natural language. Google Gemini analyzes the complaint to recommend the exact medical specialty, automatically filters registered doctors, and facilitates slot booking. Patients can also upload clinical history files to receive bilingual AI summaries, view past prescriptions with speech playback, and scan medicine packaging.
- **Doctor Portal:** Empowers consulting clinicians with a dedicated **AI Medical Briefer** tab. Doctors select authorized patients connected through existing appointments and generate multi-section clinical briefs summarizing past conditions, prescriptions, test observations, and medical timelines in English and Hindi. Clinicians can manage appointments, propose reschedules, generate one-click Google Meet video rooms, and issue digital prescriptions with automatic PDF and DOCX file generation.
- **Bilingual Voice Support:** Universal integration of the HTML5 Web SpeechSynthesis API enabling clear text-to-speech (TTS) in English (en-IN) and Hindi (hi-IN) across prescriptions and AI briefings with zero audio overlap.

2. Technology Stack

Layer	Technology	Exact Role & Implementation Purpose
Frontend Framework	React 19 + Vite	Single Page Application (SPA) architecture with fast HMR, client-side routing, and modular components
Routing & Layout	react-router-dom v7	Protected role-based nested routing for /patient/* and /doctor/* dashboard layouts
Icons & UI	lucide-react	Consistent clean vector iconography across all dashboard panels and action buttons
HTTP Client	Axios v1.19	Configured central instance with request interceptors auto-attaching Bearer JWT tokens and handling 401 unauth
Backend Framework	Python Flask + CORS	Modular REST API structured with Flask Blueprints (auth_bp, patient_bp, doctor_bp)
Authentication	PyJWT + Werkzeug	Stateless cryptographic JWT token issuance, password scrypt hashing, and decorator-based route guards
Generative AI	google-genai (Gemini 2.0/Flash)	Powers natural-language triage specialty classification, patient history synthesis, and doctor clinical briefs
OCR & Vision AI	NVIDIA Nemotron OCR v2	Cloud vision endpoint extracting text from packaging; paired with regex salt lookup in med_salts.py
Document Gen	ReportLab + python-docx	Server-side dynamic compilation of clinical prescription assets into downloadable PDF and DOCX files
Telehealth Meet	Google Calendar API v3	Automated creation of scheduled Google Meet conferences with OAuth2 token persistence
Data Storage	Flat-File JSON Storage	Thread-safe atomic read/write abstraction (storage.py) managing JSON entities in backend/data/
Voice / TTS	Web SpeechSynthesis API	Browser-native audio synthesizer hook supporting Indian English (en-IN) and Hindi (hi-IN)

3. Complete Project Directory Structure

The project follows a clean decoupled structure where the root directory contains the React Vite application and the backend/ folder houses the Python microservice.

```
MediBridge-MERGED/
├── package.json           # Frontend npm manifest & scripts (dev, build, lint)
├── vite.config.js         # Vite bundler configuration & React plugin
├── index.html             # HTML root entry point with Google Fonts
├── src/
│   ├── main.jsx           # React root rendering App inside StrictMode
│   ├── App.jsx            # Root router with ProtectedRoute trees for Patient & Doctor
│   ├── App.css            # Unified design system variables (colors, cards, badges)
│   ├── api/
│   │   ├── client.js      # Axios instance with Bearer JWT interceptor & 401 listener
│   │   └── services.js    # Modular API service methods (authApi, patientApi, doctorApi, directApi)
│   ├── context/
│   │   └── AuthContext.jsx # Authentication context provider (token, user, login, logout)
│   ├── components/
│   │   └── ProtectedRoute.jsx # Route guard validating authentication state & role permissions
│   └── hooks/
```

```

■ ■ ■■■ useSpeech.js          # Reusable bilingual SpeechSynthesis hook (en-IN/hi-IN, stop, toggle)
■ ■■■ layouts/
■ ■ ■■■ PatientLayout.jsx      # Patient sidebar navigation shell with active route highlights
■ ■ ■■■ DoctorLayout.jsx      # Doctor sidebar navigation shell including AI Briefer tab
■ ■■■ pages/
■ ■■■ LoginRegister.jsx       # Unified tabbed authentication screen (Login & Register for both roles)
■ ■■■ patient/
■ ■ ■■■ PatientDashboard.jsx   # Patient overview: metrics, upcoming consultations, quick actions
■ ■ ■■■ DoctorList.jsx        # AI Doctor Finder (symptom input + TTS) + manual specialty filter
■ ■ ■■■ PatientAppointments.jsx # Booking history, status badges, reschedule acceptance/rejection
■ ■ ■■■ PatientPrescriptions.jsx # Prescriptions archive with bilingual detail modal, TTS & downloads
■ ■ ■■■ AiBriefing.jsx         # Patient report upload & Gemini health summary generator with TTS
■ ■ ■■■ OcrScanner.jsx         # Medicine packaging scanner using NVIDIA Nemotron OCR endpoint
■ ■ ■■■ MedicalRecords.jsx     # Diagnostic PDF document manager with upload and preview
■ ■ ■■■ PatientProfile.jsx     # Patient demographic and contact info management
■ ■■■ doctor/
■ ■ ■■■ DoctorDashboard.jsx    # Doctor overview: schedule counts, pending requests, quick shortcuts
■ ■ ■■■ DoctorAppointments.jsx # Clinical schedule: Google Meet generator, prescription writer
■ ■ ■■■ DoctorRequests.jsx     # Inbound patient booking requests: Accept or propose new slot
■ ■ ■■■ DoctorAiBriefer.jsx    # AI Medical Briefer: authorized patient selector & structured summary
■ ■ ■■■ DoctorPrescriptions.jsx # Written prescriptions archive with bilingual viewer & downloads
■ ■ ■■■ DoctorProfile.jsx      # Doctor professional credentials, specialization & clinic location
■■■ backend/
■■■ index.py                   # Main Flask server entry point, CORS configuration & blueprint registration
■■■ requirements.txt           # Python dependency specifications
■■■ config.py                  # Environment config resolver & secret definitions
■■■ storage.py                 # File I/O abstraction managing JSON collections with threading locks
■■■ briefer.py                 # Google Gemini prompt engineering for patient history summarization
■■■ med_salts.py               # Medicine database & regex parser matching active pharmaceutical salts
■■■ meeting_generator.py       # Google Calendar API integration generating Google Meet video links
■■■ .env / .env.example        # Environment secret variables (Google API Key, NVIDIA OCR, Secret Key)
■■■ data/
■ ■■■ users.json               # User authentication records (passwords bcrypt-hashed)
■ ■■■ patients.json            # Patient demographics and uploaded medical document references
■ ■■■ doctors.json             # Doctor professional profiles, ratings, and specializations
■ ■■■ appointments.json        # Consultation booking records, proposed times, and Meet links
■ ■■■ prescriptions.json       # Digital prescription records with structured medicine arrays
■■■ routes/
■ ■■■ auth.py                  # Authentication endpoints: /login and /register
■ ■■■ auth_utils.py            # Token decorator guards (@token_required, @doctor_required)

```

```
■ ■■■ patient.py          # Patient endpoints: /me, /appointments, /doctors, /ai-recommend-s
pecialty
■ ■■■ doctor.py          # Doctor endpoints: /profile, /appointments, /my-patients, /ai-bri
ef
■■■ utils/
■ ■■■ prescription_generator.py # ReportLab and python-docx dynamic prescription file builders
■■■ history/             # Uploaded patient clinical history files for AI analysis
■■■ prescriptions/       # Generated prescription PDF and DOCX files
■■■ uploads/             # Stored patient medical diagnostic PDFs
```

4. File-by-File Technical Reference

This section documents every core source-code file in the codebase, detailing its responsibility, exported symbols, internal logic, and exact situations warranting modification.

File: `src/hooks/useSpeech.js`

- **Purpose:** Universal custom React hook providing browser-native Text-to-Speech capabilities.
- **Exported Symbols / Contains:** `useSpeech()` hook, `speak(text, lang)`, `stop()`, `toggle(text, lang)`, `speaking`, `currentLang` state
- **Dependencies:** Browser `window.speechSynthesis` API, React `useState/useCallback/useEffect/useRef`
- **Used By:** `DoctorList.jsx`, `AiBriefing.jsx`, `DoctorAiBriefer.jsx`, `PatientPrescriptions.jsx`, `DoctorPrescriptions.jsx`
- **Key Implementation Logic:** Preloads voices asynchronously on mount. Prioritizes 'hi-IN' for Hindi and 'en-IN' for English. Ensures strictly zero speech overlap by invoking `window.speechSynthesis.cancel()` before triggering any new utterance. Automatically cancels active speech when components unmount or pages change.
- **Modify When:** *Modify when adjusting speech rate, pitch, default voice selection priorities, or adding custom speech controls (pause/resume).*

File: `src/pages/patient/DoctorList.jsx`

- **Purpose:** Patient doctor discovery and appointment booking page featuring dual-mode manual and AI-powered specialist search.
- **Exported Symbols / Contains:** `DoctorList` component, `handleAiRecommend()`, `handleBookSubmit()`, filter logic, booking modal
- **Dependencies:** `patientApi` (`services.js`), `useSpeech` hook, `lucide-react` icons
- **Used By:** Rendered at route `/patient/doctors` via `PatientLayout`
- **Key Implementation Logic:** Renders an AI Doctor Finder prompt box where patients submit natural-language symptoms. Dispatches to `patientApi.aiRecommendSpecialty()`, receives recommended specialty + bilingual rationale, and automatically updates the specialty filter state to instantly show matching doctors. Preserves the full manual drop-down filter and appointment proposal dialog modal.
- **Modify When:** *Modify when changing doctor card layouts, rating displays, booking time-slot options, or AI suggestion display styles.*

File: `src/pages/doctor/DoctorAiBriefer.jsx`

- **Purpose:** Doctor clinical briefing dashboard facilitating authorized patient medical history review and Gemini-powered briefing synthesis.
- **Exported Symbols / Contains:** `DoctorAiBriefer` component, `handleGenerateBrief()`, `getBriefFullText()`, collapsible history preview, bilingual brief renderer
- **Dependencies:** `doctorApi` (`services.js`), `useSpeech` hook, `lucide-react` icons
- **Used By:** Rendered at route `/doctor/ai-briefer` via `DoctorLayout`
- **Key Implementation Logic:** Fetches authorized patients via `doctorApi.getMyPatients()`. On selection, loads historical records. Dispatches `doctorApi.generateAiBrief(patientId)` to call the backend Gemini summarizer. Renders 7 structured clinical sections (Summary, Conditions, Prescriptions, Reports, Observations, Key Points, Timeline) with an English/Hindi toggle and section-level TTS.

- **Modify When:** *Modify when altering clinical briefing section schemas, adding export-to-PDF options, or customizing clinician alert disclaimers.*

File: `src/pages/patient/PatientPrescriptions.jsx` & `DoctorPrescriptions.jsx`

- **Purpose:** Prescription archive interfaces allowing patients and doctors to view details, listen to prescriptions bilingually, and download files.
- **Exported Symbols / Contains:** Prescription table, `handleDownload()`, `getPrescriptionSpeechText()`, prescription detail modal with TTS
- **Dependencies:** `patientApi` / `doctorApi` (`services.js`), `useSpeech` hook, `lucide-react` icons
- **Used By:** Rendered at `/patient/prescriptions` and `/doctor/prescriptions`
- **Key Implementation Logic:** Fetches prescription list. Clicking 'View & Listen' opens a modal with complete medicine items (dosage, frequency, duration, instructions), doctor diagnosis, advice, and follow-up date. Includes an English/Hindi audio button that reads the entire prescription using the `useSpeech` hook.
- **Modify When:** *Modify when changing prescription table columns, modal layout, or medicine formatting.*

File: `backend/routes/patient.py`

- **Purpose:** Flask Blueprint handling all patient-specific REST endpoints and data mutations.
- **Exported Symbols / Contains:** `get_my_profile`, `update_my_profile`, `upload_medical_documents`, `get_medical_documents`, `create_appointment`, `get_my_appointments`, `accept_suggested_appointment`, `reject_suggested_appointment`, `get_my_prescriptions`, `download_prescription_pdf/docx`, `get_all_doctors`, `ai_recommend_specialty`
- **Dependencies:** `Flask`, `storage.py`, `routes.auth_utils.token_required`, `google.genai`, `json`, `uuid`
- **Used By:** Registered in `index.py` under prefix `/api/patients`
- **Key Implementation Logic:** Implements strict patient authentication. Route `/ai-recommend-specialty` reads active doctor specializations from `doctors.json`, prompts Gemini 2.0 to classify the patient's symptom text into one of the available specializations, and returns the specialty along with bilingual explanations.
- **Modify When:** *Modify when adding patient profile fields, changing appointment status transitions, or altering AI triage prompts.*

File: `backend/routes/doctor.py`

- **Purpose:** Flask Blueprint managing doctor schedules, appointment approvals, meeting creation, prescriptions, and AI briefings.
- **Exported Symbols / Contains:** `get_doctor_profile`, `update_doctor_profile`, `get_doctor_appointments`, `accept_appointment`, `create_appointment_meet`, `suggest_appointment_time`, `create_prescription`, `get_doctor_prescriptions`, `get_my_patients`, `get_patient_history`, `generate_ai_brief`
- **Dependencies:** `Flask`, `storage.py`, `auth_utils` (`token_required`, `doctor_required`), `prescription_generator.py`, `meeting_generator.py`, `google.genai`
- **Used By:** Registered in `index.py` under prefix `/api/doctors`
- **Key Implementation Logic:** Enforces doctor ownership on all appointments. `/my-patients` returns unique patients connected via appointments. `/patient/history` and `/patient/ai-brief` verify that the requesting doctor has a legitimate appointment with the patient before reading data or invoking Gemini for structured synthesis.

- **Modify When:** *Modify when updating doctor profile schema, prescription creation fields, appointment proposal logic, or doctor brief prompts.*

File: backend/briefer.py

- **Purpose:** AI health report summarization engine powered by Google Gemini SDK.
- **Exported Symbols / Contains:** `_get_client()`, `empty_result()`, `summarize_health_report(report_text, file_paths)`
- **Dependencies:** `google.genai`, `dotenv`, `json`, `os`
- **Used By:** `index.py` (/brief_assist endpoint for patient AI health briefing)
- **Key Implementation Logic:** Uploads patient clinical history files (.pdf, .png, .jpg) to Gemini files API, constructs a bilingual prompt instructing Gemini to return JSON with 6 specific fields (summary, duration, purpose, instruction, precaution, medicines) in English and Hindi without hallucinations.
- **Modify When:** *Modify when changing the patient health report prompt, supported file types, or JSON output fields.*

File: backend/utills/prescription_generator.py

- **Purpose:** Automated clinical prescription document compiler generating binary PDF and DOCX assets.
- **Exported Symbols / Contains:** `generate_prescription_files(prescription)`, `generate_prescription_pdf()`, `generate_prescription_docx()`
- **Dependencies:** `reportlab` (SimpleDocTemplate, Paragraph, Table, Spacer), `docx` (Document, Inches, Pt), `os`, `uuid`
- **Used By:** `doctor.py` (invoked automatically during `create_prescription`)
- **Key Implementation Logic:** Takes structured prescription dictionary, builds styled medical letterhead with MediBridge branding, doctor info, patient metadata, medicine table, clinical advice, and signature line. Saves both .pdf and .docx files in `backend/prescriptions/` and returns their relative paths.
- **Modify When:** *Modify when altering prescription document styling, adding clinic logos, or adjusting table layouts.*

File: backend/storage.py

- **Purpose:** Thread-safe data persistence layer for flat-file JSON collections.
- **Exported Symbols / Contains:** `read_data(filename)`, `write_data(filename, data)`, `threading.Lock`
- **Dependencies:** `json`, `os`, `threading`
- **Used By:** `auth.py`, `patient.py`, `doctor.py`
- **Key Implementation Logic:** Resolves absolute paths in `backend/data/`. Locks file operations with `threading.Lock` to prevent race conditions during concurrent requests. Automatically initializes empty collections if files are missing.
- **Modify When:** *Modify when migrating from flat JSON files to a relational database (PostgreSQL/MySQL/SQLite) or MongoDB.*

5. Complete Backend API Route Reference

The following table documents every active REST endpoint exposed by the Flask backend application.

Method & Route	Auth & Role	Input (Payload / Query)	Output & Purpose
POST /api/auth/register	Public	name, email, password, role ('patient' 'doctor')	Creates user record & role entity; returns 201
POST /api/auth/login	Public	email, password	Validates credentials; returns JWT token + user profile
GET /api/patients/me	JWT (Patient)	None	Returns current patient demographic details
PUT /api/patients/me	JWT (Patient)	age, gender, phone, address	Updates patient demographics in patients.json
GET /api/patients/doctors	JWT (Patient)	None	Returns full directory of registered doctors
POST /api/patients/ai-recommend-specialty	JWT (Patient)	{ symptoms: string }	Gemini evaluates problem -> returns specialty + EN/HI rationale
POST /api/patients/appointments	JWT (Patient)	doctor_id, date, time	Creates Pending appointment request
GET /api/patients/appointments	JWT (Patient)	None	Returns all appointments for logged-in patient
PUT /api/patients/appointments//accept	JWT (Patient)	None	Accepts doctor's proposed rescheduled date/time
PUT /api/patients/appointments//reject	JWT (Patient)	None	Rejects doctor's proposed reschedule -> Cancelled
POST /api/patients/me/medical-documents	JWT (Patient)	multipart/form-data: files (.pdf)	Saves diagnostic documents in uploads/ folder
GET /api/patients/me/medical-documents	JWT (Patient)	None	Returns patient's uploaded PDF document metadata
GET /api/patients/prescriptions	JWT (Patient)	None	Returns prescriptions written for this patient
GET /api/patients/prescriptions//pdf	JWT (Patient)	None	Streams binary PDF prescription download
GET /api/patients/prescriptions//docx	JWT (Patient)	None	Streams binary DOCX prescription download
GET /api/doctors/profile	JWT (Doctor)	None	Returns doctor's professional profile
PUT /api/doctors/profile	JWT (Doctor)	specialization, description, location, experience	Updates doctor credentials & practice details
GET /api/doctors/appointments	JWT (Doctor)	None	Returns appointments assigned to logged-in doctor
PUT /api/doctors/appointments//accept	JWT (Doctor)	None	Confirms pending appointment request
PUT /api/doctors/appointments//suggest	JWT (Doctor)	{ date: string, time: string }	Proposes alternate consultation time (Reschedule Proposed)
POST /api/doctors/appointments//meet	JWT (Doctor)	None	Creates Google Meet room via Google Calendar API
POST /api/doctors/appointments//prescription	JWT (Doctor)	diagnosis, medicines[], advice, follow_up_date	Generates PDF/DOCX assets and saves prescription record

GET /api/doctors/prescriptions	JWT (Doctor)	None	Returns all prescriptions authored by this doctor
GET /api/doctors/my-patients	JWT (Doctor)	None	Returns unique patients connected via appointments
GET /api/doctors/patient//history	JWT (Doctor - Auth)	None	Returns patient history (guarded by appointment check)
POST /api/doctors/patient//ai-brief	JWT (Doctor - Auth)	None	Synthesizes 7-section clinical brief via Gemini (EN/Hi)
POST /upload	Public / ID	query ?fname=ID, multipart files	Saves patient history files into history/ folder
GET /brief_assist	Public / ID	query ?fname=ID	Calls Gemini briefer on history/ files for patient summary
POST /api/extract	Public	multipart image: file	NVIDIA Nemotron OCR extraction & salt matching

6. Key System Workflows

Patient Journey & AI Doctor Finder Flow

1. **Authentication:** Patient registers/logs in via LoginRegister.jsx -> Receives JWT -> Stored in localStorage.
2. **Option A (Manual Discovery):** Patient visits DoctorList.jsx -> Filters by specialty dropdown -> Views real-time registered doctors -> Clicks 'Book Appointment'.
3. **Option B (AI Doctor Finder):** Patient inputs natural language complaint (e.g. *'Frequent itchy red rashes on forehead'*) -> Triggers patientApi.aiRecommendSpecialty() -> Backend sends symptoms + available doctor specialties to Gemini -> Gemini classifies as 'Dermatologist' with bilingual rationale -> Frontend updates specialty filter state to 'Dermatologist' -> Filtered doctors appear -> Patient can listen to reasoning in English/Hindi via useSpeech -> Patient books slot.
4. **Slot Booking:** Submits proposed date and time -> appointments.json records request with status 'Pending'.

Doctor Consultation & AI Medical Briefer Flow

1. **Dashboard & Inbound Requests:** Doctor checks DoctorRequests.jsx to accept appointments or propose alternate times ('Reschedule Proposed').
2. **AI Medical Briefer:** Doctor navigates to DoctorAiBriefer.jsx -> Dropdown lists patients with confirmed/pending appointments via doctorApi.getMyPatients().
3. **Brief Generation:** Doctor clicks 'Generate AI Brief' -> doctor.py verifies doctor-patient relationship, bundles past prescriptions and appointment logs, and prompts Gemini 2.0 -> Structured JSON response returned with 7 distinct clinical sections in both English and Hindi.
4. **Voice Playback:** Doctor toggles language (English | ■■■■■) and clicks 'Listen' -> useSpeech.js streams synthesis via SpeechSynthesisUtterance.
5. **Consultation & Prescription:** Doctor creates instant Google Meet via meeting_generator.py, conducts call, and writes prescription -> Backend compiles downloadable PDF and DOCX files.

7. Database Entity Schemas (backend/data/*.json)

```
// users.json
{ "id": "uuid", "name": "string", "email": "string", "password": "script_hash", "role": "patient|doctor" }

// patients.json
{ "id": "uuid", "name": "string", "email": "string", "age": 30, "gender": "Male", "phone": "string", "address": "string", "medical_documents": [{ "id": "uuid", "original_name": "report.pdf", "filename": "uuid.pdf", "path": "uploads/uuid.pdf" }] }

// doctors.json
{ "id": "uuid", "name": "string", "email": "string", "specialization": "Cardiologist", "description": "string", "location": "string", "experience": 8, "rating": 5.0, "available_slots": [] }

// appointments.json
{ "id": "uuid", "patient_id": "uuid", "doctor_id": "uuid", "doctorName": "string", "specialist": "string", "date": "YYYY-MM-DD", "time": "10:00 AM", "status": "Pending|Confirmed|Reschedule Proposed|Cancelled", "join_url": "https://meet.google.com/...", "event_id": "string" }

// prescriptions.json
{ "id": "uuid", "appointment_id": "uuid", "doctor_id": "uuid", "doctor_name": "string", "specialization": "string", "patient_id": "uuid", "patient_name": "string", "date": "YYYY-MM-DD", "diagnosis": "string", "medicines": [{ "name": "string", "dosage": "string", "frequency": "string", "duration": "string", "instructions": "string" }], "advice": "string", "follow_up_date": "string", "pdf": "prescriptions/uuid.pdf", "docx": "prescriptions/uuid.docx" }
```

8. Developer Practical Modification Guide

When extending or refactoring MediBridge, refer to this exact file-mapping directory:

1. Change Patient Dashboard UI / Statistics

- **Action:** Modify `src/pages/patient/PatientDashboard.jsx` and CSS in `src/App.css`.

2. Change Doctor Dashboard UI / Actions

- **Action:** Modify `src/pages/doctor/DoctorDashboard.jsx` and `src/layouts/DoctorLayout.jsx`.

3. Modify AI Doctor Recommendation Prompt

- **Action:** Modify `backend/routes/patient.py` under route `ai_recommend_specialty` (Gemini prompt string).

4. Adjust AI Doctor Finder Frontend UI / Styling

- **Action:** Modify `src/pages/patient/DoctorList.jsx` (AI Doctor Finder card section).

5. Customize Doctor AI Medical Briefer Sections

- **Action:** Modify prompt in `backend/routes/doctor.py` (`generate_ai_brief`) and section renderers in `src/pages/doctor/DoctorAiBriefer.jsx`.

6. Modify Voice / TTS Behavior or Voices

- **Action:** Modify `src/hooks/useSpeech.js` (language tag matching, pitch, rate, or pause/resume logic).

7. Alter Prescription PDF / DOCX Layout & Branding

- **Action:** Modify `backend/utills/prescription_generator.py` (ReportLab styles, table headers, document margins).

8. Change Prescription Data Fields (e.g., adding lab tests)

- **Action:** Update `DoctorAppointments.jsx` (form inputs), `doctor.py` (`create_prescription`), and `PatientPrescriptions.jsx` (modal viewer).

9. Modify User Authentication & Token Lifetimes

- **Action:** Modify `backend/routes/auth.py` (JWT payload, expiration `timedelta`) and `backend/routes/auth_utils.py`.

10. Add a New Protected Backend Endpoint

- **Action:** Create route in `backend/routes/patient.py` or `doctor.py`, decorate with `@token_required` (and `@doctor_required` if doctor), and add corresponding method in `src/api/services.js`.

9. Setup & Local Execution Guide

Follow these exact verified commands to launch the complete MediBridge stack locally.

Terminal 1 — Backend (Flask API on Port 5000)

```
# 1. Navigate to backend directory
cd backend

# 2. Activate virtual environment (Windows PowerShell)
.\venv\Scripts\Activate.ps1
# Or on Windows Command Prompt: venv\Scripts\activate.bat
# Or on macOS/Linux: source venv/bin/activate

# 3. Verify / Install Dependencies
pip install -r requirements.txt

# 4. Start Flask Backend Server
python index.py
# Server runs on: http://localhost:5000 (or http://127.0.0.1:5000)
```

Terminal 2 — Frontend (React Vite on Port 5173)

```
# 1. From repository root directory
# Ensure Node.js v18+ is installed

# 2. Install dependencies (if not already installed)
npm install

# 3. Start Vite Development Server
npm run dev
# App runs on: http://localhost:5173
```

Environment Variables Configuration (backend/.env)

Variable Name	Required?	Purpose & Service
SECRET_KEY	Yes	Cryptographic signing key for PyJWT authentication tokens
GOOGLE_API_KEY	Yes	Google Gemini API Key powering AI Doctor Finder, Patient Health Brief, and Doctor AI Briefer
NVIDIA_API_KEY	Optional	API key for NVIDIA Nemotron OCR v2 label scanner endpoint
NVIDIA_OCR_URL	Optional	NVIDIA OCR cloud endpoint URL (defaults to nemotron-ocr-v2)